

■ Documentação — Análise de Complexidade em Python

Este projeto contém três exemplos de algoritmos escritos em Python, cada um ilustrando um tipo diferente de complexidade algorítmica ($O(1)$, $O(n)$, $O(n^2)$).

■ Código 1 — `get_first_element(lista)`

```
def get_first_element(lista):  
    """  
     $O(1)$  - tempo constante  
    Retorna o primeiro elemento de uma lista (ou None se vazia).  
    """  
    if not lista:  
        return None  
    return lista[0]
```

Complexidade: $O(1)$ — Constante

■ Código 2 — `soma_elementos(lista)`

```
def soma_elementos(lista):  
    """  
     $O(n)$  - tempo linear  
    Soma todos os elementos da lista.  
    """  
    soma = 0  
    for numero in lista:  
        soma += numero  
    return soma
```

Complexidade: $O(n)$ — Linear

■ Código 3 — `pares_possiveis(lista)`

```
def pares_possiveis(lista):  
    """  
     $O(n^2)$  - tempo quadrático  
    Gera todos os pares (i, j) entre elementos da lista.  
    """  
    pares = []  
    n = len(lista)  
    for i in range(n):  
        for j in range(n):  
            pares.append((lista[i], lista[j]))  
    return pares
```

Complexidade: $O(n^2)$ — Quadrática

■ Tabela Comparativa de Complexidade

Função	Descrição	Complexidade	Tipo de crescimento
<code>get_first_element</code>	Acessa o primeiro elemento da lista	$O(1)$	Constante
<code>soma_elementos</code>	Soma todos os elementos de uma lista	$O(n)$	Linear
<code>pares_possiveis</code>	Gera todos os pares possíveis	$O(n^2)$	Quadrática

■ Conclusão

Esses três exemplos mostram como o tempo de execução de um algoritmo cresce conforme o tamanho da entrada: - $O(1)$ → sempre o mesmo tempo. - $O(n)$ → tempo proporcional à quantidade de elementos. - $O(n^2)$ → tempo cresce quadraticamente conforme o número de elementos aumenta.